

ASP.NET CORE

8. Recibiendo datos de un formulario

Podemos pasar datos desde un formulario (escrito en una vista) a la correspondiente acción del controlador. Primero veremos como hacerlo sin usar ninguna librería de asp.net y después utilizaremos el helper HTML.

Ahora nuestra etiqueta <form> quedaría de la siguiente manera

```
<form asp-controller="Controlador" asp-action="Vista" >
```

Para recoger los datos que nos llegan, crearemos una clase con los mismos atributos que controles tiene el formulario. En el controlador definiremos una vista a la que pasaremos un objeto de este tipo

Verbos de acción

El selector de acción es el atributo que se puede aplicar a los métodos de acción. Ayuda al motor de enrutamiento a seleccionar el método de acción correcto para manejar una solicitud en particular.

El selector ActionVerbs se utiliza cuando queremos controlar que método de acción se ejecuta dependiendo de la solicitud Http que le llega al controlador.

Por ejemplo, para definir dos métodos de acción diferentes con el mismo nombre, pero uno responde a una solicitud HTTP Get y el otro responde a una solicitud HTTP Post.

MVC framework admite diferentes ActionVerbs, como HttpGet, HttpPost, HttpPut, HttpDelete, HttpOptions y HttpPatch.

Si no aplica ningún atributo, entonces se considera una solicitud GET de forma predeterminada.

Patrón POST-Redirección-GET

Hay que tener mucho cuidado con la redirección. Si estando en una página, refrescamos con F5



El navegador me avisa que repetirá todas las acciones realizadas asociadas a esa URL. Supongamos que estamos en una página en la que hemos añadido algún registro a una bbdd. Esto implicaría volver a añadir con el consiguiente error si duplicamos campo clave o con la repetición de datos en la bbdd si dejamos añadir. Si estamos comprando, compraríamos dos veces lo mismo.

Para solucionar esto, este patrón lo que hace es una vez resuelta con éxito un acción POST nosotros redirigimos a una acción get. De esta forma no se vuelve a enviar el formulario.

En nuestro ejemplo, la acción POST Contact retorna una vista que sólo muestra datos por lo que refrescar no sería peligroso. Supongamos que al pulsar enviar en el formulario, ese mensaje se introduce en una bbdd. En este caso, al refrescar en la vista Contact se volvería a añadir el mensaje.....

Para solucionar esto, lo que tenemos que hacer es retornar una acción en lugar de una vista. Al redirigir a una acción que a su vez carga una vista con petición GET si refrescamos ya no se produce el reenvio.

Al ser una petición GET, todos los datos que pasamos aparecen en la URL. Si no queremos que se así, tenemos que usar la colección TempData. Ahora bien, por defecto sólo podremos almacenar valores de tipo String. ¿Qué hacemos si queremos pasar un objeto?

Podemos implementar una clase “Extendida” utilizando un paquete Nuget System.XML.XmlSerializer.

Lo veremos más adelante

De momento, podemos generar un elemento en la colección TempData con cada una de las propiedades del objeto que vamos a pasar entre acciones.

Otros Controles de un formulario

Veamos como usar los controles más frecuentes en un formulario. Ya hemos visto como funcionan los input, button, <a>, label. Veamos otros:

RadioButton

Todos los que pertenecen a un mismo grupo tienen el mismo nombre. Desde la vista al controlador, le llega a la correspondiente acción un string que contiene el value del seleccionado.

CheckBox

Si está seleccionado, se pasa un string con el value. Si no lo está se pasa null.

Si hay varios, deben llamarse igual y se pasa un string[] con los value de los seleccionados

Select

Se pasa un string con el value de la opción seleccionada.

En estos casos, los valores están predefinidos en HTML pero no es la única opción, podemos obtenerlos desde un enumerado o desde el controlador.

Veamos un par de ejemplos EjemploControlesHTML

Validación

Nunca hay que confiar en los datos que nos llegan desde el cliente. Es importante validarlos.

ASP.NET MVC usa "atributos de anotaciones" de datos para implementar validaciones.

DataAnnotations incluye clases de atributos de validación incorporados para diferentes reglas de validación, que se pueden aplicar a las propiedades del modelo. ASP.NET MVC aplicará automáticamente estas reglas y mostrará los mensajes de validación en la vista.

La siguiente tabla muestra los atributos de DataAnnotations incluidos en el espacio de nombres *System.ComponentModel.DataAnnotations*.

Attribute	Description
Required	Indicates that the property is a required field
StringLength	Defines a maximum length for string field
Range	Defines a maximum and minimum value for a numeric field
RegularExpression	Specifies that the field value must match with specified Regular Expression
CreditCard	Specifies that the specified field is a credit card number
CustomValidation	Specified custom validation method to validate the field
EmailAddress	Validates with email address format
FileExtension	Validates with file extension
MaxLength	Specifies maximum length for a string field
MinLength	Specifies minimum length for a string field
Phone	Specifies that the field is a phone number using regular expression for phone numbers

Los pasos a dar son los siguientes:

1. Incluir las reglas de validación en el modelo. Tendremos que importar el namespace DataAnnotations

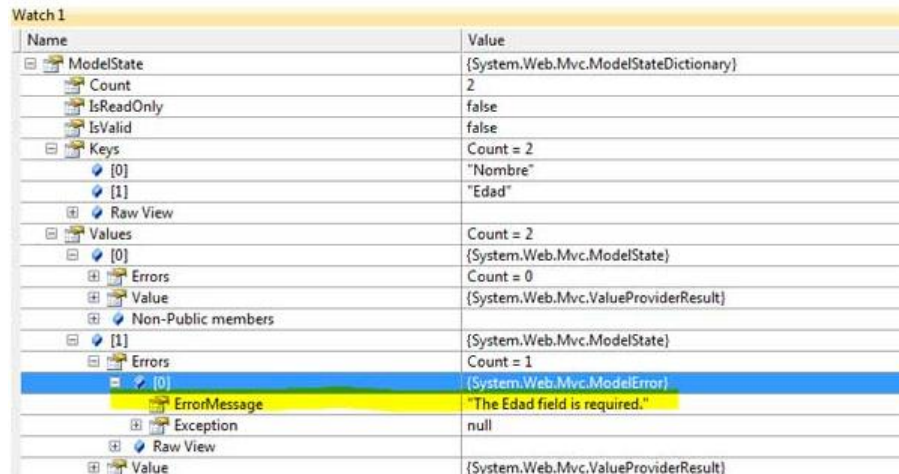
```
public class Student
{
    public int StudentId { get; set; }

    [Required]
    public string StudentName { get; set; }

    [Range(5,50)]
    public int Age { get; set; }
}
```

2. Crear los métodos de acción correspondientes en el controlador utilizando la propiedad `IsValid` del objeto `ModelState`.

Es un objeto generado automáticamente por el framework que nos indica el estado del modelo. El estado puede ser correcto o incorrecto. Por correcto entendemos que se ha generado correctamente a partir de los datos de la petición y el Model Binding ha podido instanciar el objeto con sus datos. Incorrecto significa que algún dato no era válido y por tanto se genera un error que se almacena en la colección `Error` del `ModelState`



Name	Value
ModelState	{System.Web.Mvc.ModelStateDictionary}
Count	2
IsReadOnly	false
IsValid	false
Keys	Count = 2
[0]	"Nombre"
[1]	"Edad"
Raw View	
Values	Count = 2
[0]	{System.Web.Mvc.ModelState}
Errors	Count = 0
Value	{System.Web.Mvc.ValueProviderResult}
Non-Public members	
[1]	{System.Web.Mvc.ModelState}
Errors	Count = 1
[0]	{System.Web.Mvc.ModelError}
ErrorMessage	"The Edad field is required."
Exception	null
Raw View	
Value	{System.Web.Mvc.ValueProviderResult}

El código en el controlador debe controlar si la validación ha sido correcta o no y actuar en consecuencia.

```
[HttpPost]
public ActionResult Edit(Student std)
{
    if (ModelState.IsValid) {

        //write code to update student

        return RedirectToAction("Index");
    }

    return View(std);
}
```

3. La vista: si no escribimos los mensajes de error el usuario no sabrá por qué le mostramos de nuevo el formulario.

Utilizamos el método `ValidationSummary`, que muestra una lista con los errores que se producen

```
<h1>Contacta con nosotros</h1>
<div asp-validation-summary="All" class="text-danger"> </div>
```

Contacta con nosotros

- The Nombre field is required.
- The Email field is required.
- The Mensaje field is required.

Podemos personalizar esos mensajes añadiendo dicho mensaje en la regla de validación de la propiedad del modelo

```
[Required(ErrorMessage="Please enter student name.")]
```

Podemos incluir en ese mensaje el nombre de la propiedad, de esta manera si la cambiamos no es necesario buscar todos los mensajes de error para cambiarlos también

Podemos especificar varias reglas para una misma propiedad

```
[Required(ErrorMessage="{0} es obligatorio")]
[StringLength(maximumLength:50,MinimumLength =10,ErrorMessage ="La longitud de {0} ha de estar entre {2} y {1}")]
0 referencias
public String Nombre { get; set; }
```

Veremos más validaciones

4. Por último nos faltaría enviar de vuelta aquellos valores que sean correctos para que el usuario no deba introducirlos de nuevo. Para ello, en la vista incluiremos el atributo `asp-for` (**Tag helper**) en cada control

< asp-for="Propiedad" del objeto>

Veáse `EjemploFormulario_Validaciones`

Para mostrar el mensaje de error al lado de cada control html, dónde queramos mostrarlo añadimos

```
<span asp-validation-for="Propiedad"></span>
```

NOTA: Validación en cliente

La validación en cliente no tiene nada que ver con la seguridad, es un tema de usabilidad (darle feedback al usuario de forma más rápida) así que **el uso de validación en cliente no inhibe de realizarla en servidor**. ASP.NET MVC la realiza siempre en servidor (**el desarrollador debe comprobar siempre el valor de la propiedad IsValid del ModelState**).

Validación en el fronted

En la vista parcial _ValidationScriptPartial.cshtml tenemos incluida la librería de jquery que nos permite hacer ésto. Por tanto, en nuestra vista añadimos esta otra vista parcial. Al final de la misma añadimos

```
@section Scripts{  
    <partial name="_ValidationScriptPartial" />  
}
```

Añadimos esta sección porque en el layout

```
<script src="~/lib/jquery/dist/jquery.min.js"></script>  
<script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"></script>  
<script src="~/js/site.js" asp-append-version="true"></script>  
@await RenderSectionAsync("Scripts", required: false)
```

Esa instrucción await significa que cuándo haya cargado los scripts anteriores, cargará los de las secciones de mis páginas. Es necesario así porque las librerías de la vista parcial ValidationScriptPartial utilizan las que se cargan antes ahí.

Es importante tener en cuenta que estas validaciones primero se producen del lado del cliente, es decir si hay errores no se ejecuta la acción del controlador y por tanto la validación del ModelState. IsValid.

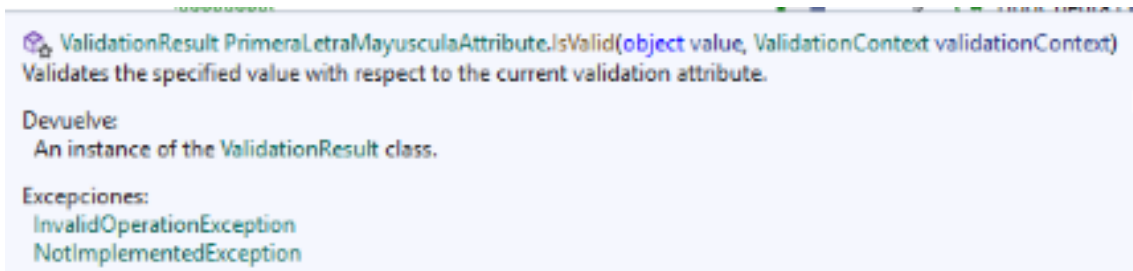
Ahora bien, en el caso de que estas validaciones sean correctas entonces sí se ejecuta la acción y por tanto la validación de nuevo pero del lado del servidor.

Véase EjemploFormularioValidaciones_2

Validaciones Personalizadas Por Atributo

Las reglas estándar que hemos visto antes pueden no ser suficientes. En ocasiones querremos controlar otras situaciones... Por ejemplo, quizás queramos que el nombre empiece por mayúsculas. Además quizás queramos usar esta validación para distintos atributos incluso de diferentes modelos. Para hacer ésto, tenemos que añadir una nueva clase en la que definiremos un método que lleve a cabo este proceso:

1. Agregamos una nueva carpeta para tener organizadas las validaciones
2. Añadimos en esa carpeta una clase que debe heredar de ValidationAttribute
3. Sobreescribimos el método IsValid



El primer parámetro será el campo recibido para validación personalizada

```
1 referencia
public class PrimeraLetraMayusculaAttribute : ValidationAttribute
{
    0 referencias
    protected override ValidationResult IsValid(object value, ValidationContext validationContext)
    {
        if (value == null || string.IsNullOrEmpty(value.ToString()))
        {
            return ValidationResult.Success;
        }

        var primeraLetra = value.ToString()[0].ToString();

        if (primeraLetra != primeraLetra.ToUpper())
        {
            return new ValidationResult("La primera letra debe ser mayúscula");
        }

        return ValidationResult.Success;
    }
}
```

Cuando queremos indicar que la validación es correcta retornamos la propiedad Success, en otro caso retornamos un error

4. Añadimos esta nueva regla de validación al atributo de la clase

```
[Required(ErrorMessage = "El campo {0} es requerido")]
[PrimeraLetraMayuscula]
```

2 referencias

```
public string? Nombre { get; set; }
```

Validaciones Personalizadas Por Modelo

Nuestro modelo debe implementar la interfaz `IValidatableObject`. Esta interfaz define el método `Validate`. En este método escribimos las reglas de validación de todos los campos que queramos.

```
public class Data:IValidatableObject
{
    [Required(ErrorMessage = "El campo {0} es obligatorio")]
    public String Nombre{get;set;}

    [Required(ErrorMessage = "El campo {0} es obligatorio")]
    [Range(18,60,ErrorMessage = "El campo {0} debe estar entre{1} - {2}")]
    public int Edad { get; set; }

    public IEnumerable<ValidationResult> Validate(ValidationContext validationContext)
    {
        //IEnumerable<ValidationResult> listError = new List<ValidationResult>();
        if (Nombre != null && Nombre.Length > 0)
        {
            string primeraLetra = Nombre[0].ToString();
            if (primeraLetra != primeraLetra.ToUpper())
            {
                // listError.Append(new ValidationResult("La primera letra en mayúsculas",
                //     new[] { nameof(Nombre) }));
                yield return new ValidationResult("La primera letra en mayúsculas",
                    new[] { nameof(Nombre) });
            }
            //Podría validar más atributos e ir añadiendo los posibles errores.
            // return listError;
        }
    }
}
```

Si al retornar no añadimos el campo que produce el error en el constructor de `ValidationResult`, entonces se considera una validación asociada al modelo y el mensaje saldrá en la vista en el lugar del `validation-summary`.

Si añadimos el campo y en `validation-summary` tenemos el valor `ModelOnly`, sólo saldrá el mensaje para el campo pero si tenemos `All` saldrán en ambos sitios.

